

Script de Presentacion: Optimizacion Inteligente de Estanterias con IA

JULIAN — Diapositivas 2, 3 y 4

Diapositiva 2: Objetivo del Proyecto

Buenas. Vamos a presentar un sistema que hemos construido para optimizar automaticamente la colocacion de productos en estanterias de supermercado. El objetivo es sencillo: conseguir que los productos esten en la posicion que maximiza el beneficio total de la tienda.

Para llegar ahi, hemos tenido que resolver cinco problemas. Primero, generar datos de ventas mensuales realistas usando un LLM, porque no teniamos un historico de ventas real sino un catalogo de productos. Segundo, entrenar y comparar cuatro modelos de machine learning distintos: MLP, LSTM, Transformer y PPO, para predecir que pasa cuando mueves un producto de un estante a otro. Tercero, construir un pipeline RAG que usa datos historicos para prever la demanda futura. Cuarto, usar la mejor combinacion de modelos para generar layouts optimizados para cualquier mes futuro. Y quinto, medir en euros cuanto beneficio extra genera el sistema respecto a la disposicion actual.

Diapositiva 3: Restricciones Iniciales

No trabajamos en el vacio. El sistema tiene que respetar restricciones reales. A la izquierda veis las restricciones fisicas: cada estante mide 300 centimetros de ancho, hay 7 niveles por rack, y los estantes a nivel de ojos, los niveles 3 y 4, venden significativamente mas que el suelo o la parte alta. Los productos tienen anchos variables, asi que no puedes meter los que quieras en un estante.

A la derecha, el contexto de datos. Partimos de un catalogo real de un supermercado espanol con aproximadamente 4.500 productos en 149 categorias. Los datos de ventas tienen que reflejar estacionalidad real: mas helado en verano, mas turrón en diciembre. Y todo el pipeline tiene que ser reproducible de principio a fin.

Diapositiva 4: Arquitectura del Pipeline

Aqui veis como se conecta todo. El sistema tiene dos fases.

En la fase de entrenamiento, el paso 01 genera 12 ficheros CSV de ventas mensuales usando el LLM. El paso 02 entrena los cuatro modelos sobre esos datos. Y el paso 03 evalua y compara los resultados.

En la fase de produccion, el paso 04 ingesta los datos en paralelo: por un lado genera embeddings para la base de conocimiento en ChromaDB, y por otro entrena los modelos MLP y Transformer. Luego el paso 05 ejecuta el pipeline completo: RAG para contexto historico, LLM para forecast de ventas, y el ensemble para optimizar la disposicion.

La salida final es un CSV con el layout optimizado, un informe de beneficios y un JSON con los multiplicadores de forecast. Con esto paso a Rulas, que os va a contar como generamos los datos.

RULAS — Diapositivas 5, 6, 7 y 8

Diapositiva 5: Generacion de Datos

Gracias, Julian. Vamos con la generacion de datos, que es el primer paso de todo el pipeline.

Para cada uno de los 12 meses del ano, seguimos cinco pasos. Primero, muestreamos entre el 60 y el 90 por ciento del catalogo, porque no todos los productos estan disponibles cada mes. Segundo, mandamos lotes de 30 productos al LLM, que nos devuelve tres valores por producto: ventas estimadas, margen de beneficio y ancho del producto en centimetros. Tercero, aplicamos factores estacionales predefinidos. Cuarto, asignamos cada producto a un rack, uno por categoria, y a un estante aleatorio del 1 al 7. Y quinto, verificamos que ningun estante supere los 300 centimetros de ancho total.

A la derecha veis las seis capas anti-alucinacion que protegen la calidad de los datos. Desde un prompt muy restringido que solo pide tres numeros, pasando por clampeo de valores a rangos realistas, validacion de longitud, extraccion JSON con regex, reintentos con fallback de modelo, hasta heurísticas locales si todas las llamadas a la API fallan. Con esto nos aseguramos de que aunque el LLM se equivoque, los datos siempre estan dentro de rangos razonables.

Diapositiva 6: Factores Estacionales

Aqui veis ejemplos concretos de los multiplicadores estacionales que aplicamos. El helado se multiplica por 1,8 en julio por el calor, el agua por 1,6 por la hidratacion. En diciembre, el chocolate se multiplica por 1,8 por los regalos navidenos y el turrón por 2,5 porque es el dulce típico de Navidad. Y en enero, la fruta baja a 0,8 porque esta fuera de temporada.

El resultado son 12 ficheros CSV mensuales, cada uno con entre 3.000 y 3.500 productos, que suman 42.301 registros producto-mes en total. La semilla es determinista, así que si ejecutas el script dos veces con la misma semilla, obtienes los mismos datos.

Diapositiva 7: Datos de Entrenamiento — Simulador Fisico

Ahora viene una parte clave: como generamos los datos de entrenamiento para los modelos. No entrenamos directamente sobre los CSV mensuales. Usamos un simulador fisico, `retail_physics.py`, que codifica como afecta la posicion del estante a las ventas.

A la izquierda teneis la formula de beneficio: precio por margen por ventas por un multiplicador que depende del estante. El estante 4, a nivel de ojos, tiene el multiplicador mas alto con 1,15. El suelo tiene 0,60 y la parte alta 0,50.

A la derecha, un ejemplo concreto. Aceite de oliva a 8,50 euros, 40% de margen, 200 ventas al mes. En el estante 2, con multiplicador 0,80, genera 544 euros. En el estante 4, con multiplicador 1,15, genera 782 euros. El `profit_lift` de ese movimiento es +238 euros. Esa diferencia es lo que los modelos aprenden a predecir.

En total generamos 126.903 muestras de entrenamiento y 42.301 de test.

Diapositiva 8: Por que un Simulador Fisico?

Y esta es la pregunta clave: por que necesitamos este simulador? Por que no entrenar directamente con los datos de ventas?

El problema es que los CSV mensuales son fotos fijas. Te dicen que el aceite de oliva esta en el estante 3 y vende 200 unidades al mes. Pero no te dicen lo mas importante: que pasaria si lo movemos al estante 5. Un CSV es una observacion, no un experimento.

La solucion es que retail_physics codifica el conocimiento de retail en una formula y genera ejemplos simulados de intercambios: mover el aceite del estante 3 al 4 da +238 euros, mover la sal del 4 al 1 da -9,90 euros, y asi con miles de combinaciones.

Por que no usamos datos reales? Porque no existen. Ningun supermercado intercambia miles de productos cada dia solo para registrar que pasa. Estos datos causa-efecto hay que simularlos.

Lo que los modelos aprenden es exactamente esto: dado un input con las features del producto, el estante actual y el propuesto, predecir el cambio exacto en beneficio del rack. Ahora Samu os va a contar como funcionan los cuatro modelos que hemos entrenado.

SAMU — Diapositivas 9, 10, 11 y 12

Diapositiva 9: 4 Arquitecturas de Modelos

Gracias, Rulas. Hemos entrenado cuatro modelos, cada uno representando un enfoque fundamentalmente distinto para resolver el problema.

El MLP es un feedforward clasico, la red neuronal mas simple. La pregunta que responde es: puede un mapeo directo de features a salida resolver esto? Con unos 53.000 parametros.

El LSTM es secuencial, procesa los productos uno a uno manteniendo memoria. La pregunta es si el orden de los productos importa. Tiene unos 45.000 parametros.

El Transformer usa mecanismos de atencion para que cada producto vea a todos los demas simultaneamente. La pregunta es si las interacciones cruzadas entre productos son relevantes. Es el modelo mas grande con unos 400.000 parametros.

Y el PPO es aprendizaje por refuerzo, aprende por ensayo y error sin necesitar etiquetas. La pregunta es si este enfoque puede superar al aprendizaje supervisado. Tiene unos 35.000 parametros.

Diapositiva 10: MLP en Detalle

Empecemos por el MLP, que resulto ser nuestro mejor optimizador. La arquitectura es sencilla: 10 features de entrada pasan por tres capas lineales con ReLU y dropout del 15%. La primera capa expande de 10 a 256 neuronas, la segunda comprime a 128, la tercera a 64, y la salida es un unico numero: el profit_lift predicho.

Lo importante del MLP es como se usa durante la optimizacion. Para cada producto, se llama 7 veces, una por cada estante posible. Y se elige el estante con el mayor profit_lift predicho. Un rack de 30 productos son 210 llamadas, que se ejecutan en microsegundos.

Sus fortalezas: es el mejor optimizador con +1.648 euros de profit lift, es extremadamente rapido, y sus decisiones son audaces y directas. Su limitacion es que no tiene conciencia inter-producto: evalua cada producto de forma independiente.

En la parte inferior veis sus numeros clave: MSE de 553, entrenado en 80 epochs, con unos 53.000 parametros.

Diapositiva 11: Transformer en Detalle

El Transformer es nuestro modelo mas preciso. La arquitectura final, la version 3, usa BatchNorm sobre las 10 features de entrada, proyeccion a dimension 128, cuatro capas de TransformerEncoder con Pre-LN y 4 cabezas de atencion, activacion GELU, y un head de salida con dos capas lineales.

Llegar a esta version no fue trivial. La version 1 con Post-LN tenia entrenamiento inestable, MSE de 1.021. La version 2 con LayerNorm mejoro a 823 pero con convergencia lenta. La version 3, con BatchNorm, Pre-LN y GELU, bajo a 299. La clave fue que las features tenian escalas muy diferentes: precio va de 0,50 a 50, y ventas de 1 a 500. Sin BatchNorm, el mecanismo de atencion estaba dominado por las ventas.

Sus fortalezas: mejor precision de prediccion con MSE 299, contexto bidireccional completo donde cada producto ve a todos los demas, y computacion paralela. Su limitacion: es un optimizador conservador, solo consigue +651 euros de profit lift cuando se usa solo, y necesita mas datos, mas epochs y un tuning cuidadoso.

Diapositiva 12: LSTM y PPO

Los otros dos modelos. El LSTM es una red recurrente que procesa productos secuencialmente manteniendo memoria de los anteriores. Tiene 2 capas apiladas con dimension 64, toma secuencias de 10 productos y devuelve 10 predicciones de profit_lift. Consiguio un MSE de 699 y +390 euros de profit lift. Su principal limitacion es que el procesamiento es unidireccional: cuando evalua el producto 5, solo ha visto los productos 1 a 4.

El PPO es aprendizaje por refuerzo. Tiene un actor que propone que dos productos intercambien sus estantes, y un critic que evalua si la accion fue buena. Lo entrenamos durante 500 episodios de 50 swaps en un unico rack. Consiguio +230 euros de profit lift, el peor resultado. La razon es que las recompensas en nuestro problema son inmediatas y calculables directamente. El refuerzo brilla cuando las recompensas son complejas y diferidas, como en ajedrez o robotica, no en este caso.

Ahora Luis os va a explicar como evaluamos y comparamos todos estos modelos.

LUIS — Diapositivas 13, 14, 15 y 16

Diapositiva 13: Metricas de Evaluacion — MSE y Profit Lift

Gracias, Samu. Antes de ver los resultados, es fundamental entender las dos metricas que usamos y por que elegimos estas y no otras.

A la izquierda, MSE, Mean Squared Error. Mide la precision con la que el modelo predice el profit_lift en euros. Se calcula como la media del cuadrado de las diferencias entre prediccion y valor real, sobre 42.301 muestras de test que el modelo nunca ha visto durante el entrenamiento. Para interpretarlo: un MSE de 553 del MLP significa que la raiz cuadrada es unos 23,5 euros de error medio por producto. Un MSE de 299 del Transformer significa unos 17,3 euros.

A la derecha, Profit Lift. Mide el beneficio real extra en euros que genera el modelo cuando toma decisiones de optimizacion. Se calcula sobre un rack real de test con 40 productos: calculas el beneficio original, dejas que el modelo optimice probando los 7 estantes por producto, calculas el nuevo beneficio, y la diferencia es el lift. El MLP consigue +1.648 euros, el Transformer +651, y el baseline greedy pierde 2.477 euros.

Y aqui viene lo clave, en la parte inferior. Necesitamos ambas metricas porque precision de prediccion no es lo mismo que calidad de decision. El Transformer predice con la mitad de error que el MLP, pero el MLP genera dos veces y media mas beneficio. Si hubieramos usado solo MSE, habriamos elegido el modelo equivocado para nuestro negocio. Y por que no usamos Accuracy, F1 o AUC? Porque son metricas de clasificacion, y nuestro problema es regresion: predecir un valor continuo en euros.

Diapositiva 14: Comparacion de Modelos

Ahora veamos los numeros. En el grafico de la izquierda teneis el profit lift de cada modelo. El MLP lidera con +1.648 euros, seguido del Transformer con +651, LSTM con +390 y PPO con +230. Fijaos en el greedy, que es el baseline de poner los productos de mayor margen a nivel de ojos: pierde 2.477 euros. Esto demuestra que una heuristica ingenua no funciona porque ignora el hacinamiento y las dependencias entre productos.

A la derecha, los MSE. Aqui el orden es inverso: el Transformer es el mejor con 299, el MLP tiene 553 y el LSTM 699.

La paradoja que veis reflejada es clara: MSE bajo no implica mejor optimizacion. La precision de prediccion no se traduce directamente en calidad de decision. Un modelo puede predecir valores inexactos pero acertar consistentemente en cual es el mejor estante. Y eso es exactamente lo que hace el MLP.

Diapositiva 15: Ensemble — MLP Propone, Transformer Valida

Esta paradoja nos llevo directamente al enfoque ensemble. La idea es simple: si el MLP es un explorador creativo que toma decisiones audaces, y el Transformer es un juez preciso que distingue matices, combinemoslos.

A la izquierda, el MLP genera 5 layouts candidatos por rack. El candidato 0 es greedy puro, sin ruido. Los candidatos 1 a 4 anaden niveles crecientes de ruido gaussiano a las predicciones del MLP antes de decidir: 5, 10, 15 y 20. Este ruido no modifica los datos del producto, solo perturba la decision de que estante elegir. Como los productos se asignan uno a uno, una pequena perturbacion al principio crea un layout completamente distinto al final.

A la derecha, el Transformer evalua cada candidato completo usando self-attention, suma el profit_lift predicho de todos los productos, y el candidato con mayor puntuacion gana. Su precision superior, MSE 299 frente a 553, le permite distinguir entre layouts buenos y ligeramente mejores.

Este patron de generate-and-rank es comun en machine learning: beam search en NLP, evaluador de trayectorias en robotica, evaluador de movimientos en juegos. Juntos consiguen +67.794 euros al mes, mucho mas que cualquiera de los dos por separado.

Diapositiva 16: Knowledge Base — Ingestion en ChromaDB

Antes de pasar a Victor, necesitais entender como construimos la base de conocimiento que alimenta todo el sistema RAG.

El script 04_ingest.py ejecuta dos hilos en paralelo que usan recursos diferentes. El hilo 1 se encarga de los embeddings: parsea cada CSV mensual por categoria, genera resmenes en lenguaje natural con las metricas agregadas de cada categoria en cada mes, los embebe con sentence-transformers y almacena los vectores en ChromaDB. El hilo 2 genera muestras de entrenamiento con retail_physics y entrena el MLP durante 80 epochs y el Transformer durante 120. Como uno usa CPU mas API y el otro CPU mas GPU, corren en paralelo y reducen el tiempo total de 18 a 10 minutos.

A la derecha arriba veis que se almacena exactamente en ChromaDB: resmenes por categoria y mes en lenguaje natural. Cada documento contiene las metricas agregadas de una categoria en un mes concreto. Por ejemplo, para yogures griegos en enero 2025: ventas medias 180 unidades, precio medio 2,40 euros, estantes mas frecuentes 3 y 4, 12 productos. En total, 149 categorias por 12 meses nos dan aproximadamente 1.788 documentos embebidos.

Abajo a la derecha, la estrategia de retrieval. Cuando queremos predecir enero 2026, recuperamos tres contextos por categoria: enero 2025 para capturar el patron estacional del mismo mes, y noviembre y diciembre 2025 para capturar la tendencia actual. Esto es lo que luego le pasamos al LLM para que genere los multiplicadores de ventas. Y ahora Victor os cuenta como se junta todo en el pipeline de prediccion.

VICTOR — Diapositivas 17, 18, 19 y cierre (20)

Diapositiva 17: Pipeline de Prediccion

Gracias, Luis. Vamos a ver como funciona el pipeline de prediccion completo, el script 05_predict.py, que junta todo lo que os han contado en un flujo de 6 pasos.

Paso 1, RAG Retrieval. Consultamos ChromaDB para recuperar el contexto historico que acabamos de ver: el mismo mes del ano anterior y los dos meses mas recientes. Paso 2, cargamos el CSV mas reciente como linea base, por ejemplo diciembre 2025 con 3.133 productos. Paso 3, mandamos todo ese contexto al LLM, que predice multiplicadores de ventas por categoria. Paso 4, multiplicamos las ventas base por esos multiplicadores. Paso 5, el ensemble: el MLP genera 5 layouts, el Transformer puntua y gana el mejor. Paso 6, guardamos el CSV optimizado, el JSON del forecast y la tabla comparativa de beneficios.

Un detalle importante que veis en la franja inferior: los datos del RAG nunca llegan a los modelos MLP y Transformer directamente. Fluyen a traves del LLM, que los convierte en ajustes numericos de ventas. Los modelos no saben que hay un LLM detras; simplemente reciben numeros distintos.

Diapositiva 18: Decisiones de Diseno Clave

Aqui recogemos las decisiones tecnicas mas importantes y su justificacion.

Usamos RAG mas LLM en vez de modelos de series temporales como ARIMA o Prophet porque solo tenemos 12 meses de datos, insuficientes para esos modelos. El LLM aporta conocimiento del mundo real que complementa nuestros datos limitados.

Elegimos Pre-LN Transformer en vez de Post-LN porque da gradientes mas estables y convergencia mas rapida; es el estandar moderno desde GPT-2. Aplicamos BatchNorm en los inputs del Transformer porque las features tienen escalas muy diferentes, y esto redujo el MSE de 823 a 299. Usamos ruido gaussiano para generar candidatos diversos porque es simple, efectivo y crea un gradiente de diversidad natural. El LLM lo configuramos con temperatura 0,3 para obtener estimaciones consistentes con ligera variacion para evitar outputs degenerados. Y la ingestion es paralela porque embeddings y entrenamiento usan recursos diferentes: CPU mas API y CPU mas GPU respectivamente, reduciendo el tiempo de 18 a 10 minutos.

Diapositiva 19: Resultados Finales

Y aqui estan los resultados. Los tres numeros grandes: +67.794 euros de beneficio mensual extra, un incremento del 16,2%, sobre 3.133 productos optimizados.

En la tabla de detalle: 149 racks cubiertos, 149 categorias, el mejor modelo de prediccion es el Transformer con MSE 299, el mejor modelo de optimizacion es el MLP con +1.648 euros de profit lift, el enfoque final es el ensemble MLP mas Transformer, y el pipeline completo es RAG mas LLM forecast mas ensemble.

Diapositiva 20: Cierre

Para cerrar: este sistema genera 67.794 euros extra al mes, un 16,2% de incremento de beneficio. El MLP propone, el Transformer valida, el LLM predice y el RAG contextualiza. Cuatro piezas que trabajan juntas para resolver un problema que una heuristica simple no solo no resuelve, sino que empeora. Gracias.